

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 1: 1. Модель данных для магазина (Пишем в самом верху, СНАРУЖИ экрана)

```
// Чертеж нашей карточки магазина. Пишем в самом верху файла.  
// Внимание: Здесь только круглые скобки '(' и ')'  
data class UpgradeItem(  
    val id: String,  
    val name: String,  
    val description: String,  
    val baseCost: Long,  
    val costMultiplier: Double,  
    val level: Int,  
    val icon: String  
) {  
    // Формула автоматического расчета цены:  
    val currentCost: Long  
    get() = (baseCost * kotlin.math.pow(costMultiplier, level.toDouble())).toLong()  
}
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 2: 2. Создаем отдельный кирпичик UI: Строка Улучшения (UpgradeRow)

```
// Пишем СНАРУЖИ главного экрана. Это отдельный мини-экран для одной строки
магазина.
// Передаем внутрь данные апгрейда и действие при нажатии кнопки 'onBuyClick'.
@Composable
fun UpgradeRow(upgrade: UpgradeItem, canAfford: Boolean, onBuyClick: () -> Unit)
{
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = Color(0xFF131A2B))
    ) {
        Row(
            modifier = Modifier.fillMaxWidth().padding(12.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(upgrade.icon + " " + upgrade.name, color = Color.White, fontWeight =
                FontWeight.Bold)
                Text(upgrade.description, color = Color.Gray, fontSize = 11.sp)
                Text("Ур. " + upgrade.level, color = Color(0xFF00FFCC), fontSize = 11.sp)
            }
            Button(onClick = onBuyClick, enabled = canAfford) {
                Text(upgrade.currentCost.toString() + " E")
            }
        }
    }
}
```

ЭТАП 3: 3. Объявляем главный экран (Голова)

```
// Всегда пишем аннотацию @Composable с большой буквы!
@Composable
fun SolarHarvesterScreen() {
    // ВСЕ КОД ИЗ СЛЕДУЮЩИХ ШАГОВ (ДО ПРЕВЬЮ) ПИШЕМ ТУТ, ВНУТРИ ЭТОЙ ФУНКЦИИ
    // Проверь, чтобы в самом конце файла стояла закрывающая фигурная скобка '}'!
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 4: 4. Переменные состояния (Память реактора)

```
// Вставляем этот код внутрь функции SolarHarvesterScreen.  
// L означает тип Long (целые числа), f означает тип Float (дробные числа)  
var energy by remember { mutableStateOf(0L) } // Сколько энергии накопили  
var isRunning by remember { mutableStateOf(false) } // Включен ли сборщик  
var heatLevel by remember { mutableStateOf(0f) } // Перегрев (0.0f - холод, 1.0f  
- взрыв)  
  
// Уровни наших улучшений  
var solarLevel by remember { mutableStateOf(1) }  
var coolingLevel by remember { mutableStateOf(1) }  
var batteryLevel by remember { mutableStateOf(1) }
```

ЭТАП 5: 5. Динамическая математика (Характеристики)

```
// Эти параметры пересчитываются автоматически при изменении уровней.  
val energyPerTick = solarLevel * 8L // Сколько энергии дает один тик  
val heatPerTick = 0.04f / (1f + (coolingLevel * 0.15f)) // Нагрев ядра  
val maxCapacity = batteryLevel * 400L // Лимит батареи
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 6: 6. Автоматический реактор (LaunchedEffect)

```
// Запускает фоновый таймер. Следи за закрытием круглых и фигурных скобок!
LaunchedEffect(isRunning) {
    if (isRunning) {
        while (isRunning) {
            delay(300) // Ждем 0.3 секунды
            heatLevel += heatPerTick

            // Проверка на взрыв реактора
            if (heatLevel >= 1.0f) {
                isRunning = false
                energy /= 2 // Наказание: теряем половину энергии
                heatLevel = 0f
            }

            // Начисляем энергию не выше лимита батареи
            if (energy < maxCapacity) {
                energy = (energy + energyPerTick).coerceAtMost(maxCapacity)
            }
        }
    }
}
```

ЭТАП 7: 7. Основной контейнер экрана (Box)

```
// Box закрашивает весь экран. Все элементы внутри него накладываются слоями.
Box(
    modifier = Modifier.fillMaxSize().background(Color(0xFF070B12)),
    contentAlignment = Alignment.Center
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        /* СЮДА БУДЕМ ВКЛАДЫВАТЬ ТЕКСТЫ И КНОПКИ */
    }
}
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 8: 8. Панель мониторов (Dashboard)

```
// Вставляем этот блок внутрь Column (Шаг 7)
Column(horizontalAlignment = Alignment.CenterHorizontally) {
    Text("КВАНТОВЫЙ РЕАКТОР", color = Color.Gray, fontSize = 12.sp)
    Spacer(Modifier.height(8.dp))
    Text(
        text = "$energy / $maxCapacity E",
        color = Color(0xFF00FFCC),
        fontSize = 32.sp,
        fontWeight = FontWeight.Bold
    )
    Spacer(Modifier.height(16.dp))
    Text("Температура ядра реактора:", color = Color.White, fontSize = 12.sp)
    Spacer(Modifier.height(4.dp))

    // Шкала перегрева ядра (краснеет при опасности)
    LinearProgressIndicator(
        progress = { heatLevel },
        modifier = Modifier.fillMaxWidth().height(12.dp).clip(RoundedCornerShape(6.dp)),
        color = if (heatLevel > 0.75f) Color.Red else Color(0xFF00FFCC)
    )
}
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 9: 9. Панель ручного контроля (Управление)

```
// Добавляем горизонтальную строку кнопок ниже панели мониторов (Шаг 8)
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    Button(
        onClick = { isRunning = !isRunning },
        modifier = Modifier.weight(1f).height(50.dp),
        colors = ButtonDefaults.buttonColors(
            containerColor = if (isRunning) Color(0xFFD32F2F) else Color(0xFF388E3C)
        )
    ) {
        Text(if (isRunning) "СТОП" else "СТАРТ", fontWeight = FontWeight.Bold)
    }
    Button(
        onClick = {
            if (heatLevel > 0f) {
                heatLevel = (heatLevel - 0.15f).coerceAtLeast(0f)
            }
        },
        modifier = Modifier.weight(1f).height(50.dp),
        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF1976D2)),
        enabled = heatLevel > 0f
    ) {
        Text("ОХЛАДИТЬ", fontWeight = FontWeight.Bold)
    }
}
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 10: 10. Создаем Магазин Улучшений через LazyColumn

```
// Список наших апгрейдов на основе текущих уровней
val upgrades = listOf(
    UpgradeItem("solar", "Солнечные Панели", "+$energyPerTick Е/тик", 100L, 1.4,
        solarLevel, "☀ "),
    UpgradeItem("heatsink", "Охлаждение радиатора", "-15% нагрева", 150L, 1.3,
        coolingLevel, "❄ "),
    UpgradeItem("battery", "Емкость Батареи", "+400 макс. энергии", 200L, 1.5,
        batteryLevel, " ")
)

// Мы используем созданный ранее кирпичик UpgradeRow! Код стал гораздо чище.
LazyColumn(
    verticalArrangement = Arrangement.spacedBy(8.dp),
    modifier = Modifier.fillMaxWidth().height(260.dp)
) {
    items(upgrades) { item ->
        UpgradeRow(
            upgrade = item,
            canAfford = energy >= item.currentCost,
            onBuyClick = {
                energy -= item.currentCost
                when (item.id) {
                    "solar" -> solarLevel += 1
                    "heatsink" -> coolingLevel += 1
                    "battery" -> batteryLevel += 1
                }
            }
        )
    }
}
```

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 11: 11. Предпросмотр Экрана (Preview)

```
// Внимание! Этот блок пишется СНАРУЖИ основной функции SolarHarvesterScreen!  
// Проверь, что закрыл все фигурные скобки экрана перед этим кодом.  
@Preview  
@Composable  
fun SolarHarvesterPreview() {  
    MaterialTheme {  
        SolarHarvesterScreen()  
    }  
}
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь Систему Сверхзаряда: создай переменную superchargeChance. Сделай так, чтобы с шансом 15% (используй Random) ручное охлаждение давало мгновенный бонус +50 энергии вместо обычного сброса тепла.